

Assessment 2

Group 10 Members:

**Ben Walker, Josh White, Chris Williams, Gavindu Tissera, Nana
Twum Oviarobo, Tamerlan Urazbayev, Tymur Topala**

Architecture

1. Architectural Style

The architecture will adopt a hybrid approach combining **MVC** and **ECS**, allowing components to handle user input and game mechanics while maintaining modular, event-driven interactions.

MVC Pattern

- **Model:** Represents the game's data (campus map, building placement, events, student satisfaction, time).
- **View:** Renders the game's grid, buildings, events, and satisfaction metre.
- **Controller:** Interprets user input (keyboard and mouse) and updates the model or view as required.

Entity-Component-System (ECS)

- **Entity:** Represents game objects like buildings, roads, or events.
- **Component:** Defines individual properties for each entity, such as building type, location, or impact on satisfaction.
- **System:** Manages interactions and logic between components, such as placing buildings on the grid, generating events, and adjusting student satisfaction based on game state.

2. Main Components

1. Game Engine Layer

The game engine is responsible for handling core mechanics like game state, event management, rendering, and user input handling. It will leverage an existing framework, such as **libGDX** or **jMonkeyEngine**, to speed up development and focus on game logic rather than reinventing basic functionality.

- **Grid-based Map:** The map is a two-dimensional grid where users can place buildings and roads (`FR_MAP`). Each grid cell contains either a building or a road, and players can modify building locations (`FR_PLACE_BUILDING`, `FR_MOVE_BUILDING`).
- **Game Timer:** A countdown timer that starts at 5 minutes, pausing when needed (`FR_TIMER`, `FR_COMPLETE_PAUSE`). The time is displayed on the user interface to keep the player informed.
- **Physics/Rendering:** Manages the real-time rendering of objects on the grid and simulates player actions, ensuring a smooth gameplay experience.

2. Map Module

The **Map Module** will represent the campus layout as a grid-based environment. It handles building placement, roads, and environmental constraints like lakes and trees that limit where structures can be placed (`UR_PLACE_BUILDING`).

- **Placement System:** Handles rules for placing buildings and moving them across the map. Buildings should be placed logically, following the game rules (FR_PLACE_BUILDING).
- **Road System:** Allows players to create roads and paths that connect buildings. The proximity and accessibility of buildings impact student satisfaction (FR_ROAD).

3. Student Satisfaction Module

This module is responsible for calculating and updating student satisfaction based on building placement, event handling, and proximity of facilities (FR_CALCULATE_SATISFACTION).

- **Satisfaction Algorithm:** Uses factors like building proximity (e.g., accommodation near lecture halls, food areas close to student zones) and event outcomes to update student satisfaction (UR_INFLUENCE_SATISFACTION).

4. Event System

The game will include both random and pre-planned events, which will influence the player's strategy in maximising satisfaction. Events may have positive, negative, or neutral effects, and players must react accordingly (FR_EVENTS).

- **Random Event Generator:** Triggers events based on certain intervals or conditions during gameplay.
- **Fixed Events:** These are predictable and happen in all playthroughs, adding consistency to the game.

5. User Interface (UI) System

The user interface provides visual feedback to the player and facilitates interaction with the game. The UI should be intuitive and responsive, allowing users to control the game through both keyboard and mouse inputs (FR_MOUSE_INPUT, FR_KEY_INPUT).

- **Grid Display:** The grid-based map is rendered clearly, with distinguishable buildings and road placements (NFR_CLEAR_GRAPHICS).
- **Hover Information:** Players can hover over buildings to view detailed information, including how it affects student satisfaction (FR_HOVER_INFO).
- **Settings Menu:** Provides options for muting, volume control, and adjusting gameplay experience (FR_MUTE, FR_ALTER_VOLUME, FR_SETTINGS).

6. Input Management

- **Flexible Input:** The system supports both keyboard and mouse controls. It must handle user inputs, whether simultaneous or separate, without conflicts (UR_FLEXIBLE_UI, UR_SHORTCUTS).
- **Keyboard Shortcuts:** Allow faster navigation and control of the game, enhancing the user experience (UR_SHORTCUTS).

3. Additional Features

Data Persistence

The system will store game data using JSON or similar formats to track progress and events. This allows easy storage of game states and retrieval across multiple platforms (`FR_EVENTS`, `FR_OPERATING_SYSTEMS`).

Modularity

Each game component is designed with high cohesion and low coupling, ensuring that individual features (like the map, satisfaction module, and event system) can be easily maintained or updated independently.

Scalability and Accessibility

The game will be accessible across different devices and screen sizes, from laptops to desktop monitors (`NFR_SCALABLE`). The interface adapts dynamically, ensuring a clear and consistent user experience.

4. Technology Stack

- **Language:** The game will use **Java 17**, compatible with frameworks like libGDX, ensuring cross-platform functionality.
- **Rendering Engine:** libGDX will handle the 2D rendering for the grid, buildings, roads, and events.
- **Input Handling:** Both keyboard and mouse input will be managed using standard input libraries from the game engine.
- **Data Management:** **JSON** or similar data storage format for events, building states, and player interactions.

5. Architecture Trade-offs

Drawing on concepts from the software architecture lecture, we must consider trade-offs to ensure the architecture supports long-term maintainability and performance:

- **Modularity vs. Performance:** Modularisation increases maintainability but may introduce slight overheads due to component communication.
- **Event-based Communication:** An event-driven approach ensures flexibility but adds complexity in managing event processing and sequencing.

This architecture provides a robust foundation for UniSim, supporting its grid-based design and gameplay mechanics. By combining the MVC and ECS patterns, we ensure a clear separation of concerns and high modularity, making it easier to expand and maintain the game over time. The architecture also addresses key requirements like accessibility, flexibility in input handling, and a user-friendly interface, all while ensuring that the core game experience - building a thriving university campus - is intuitive and engaging.

6. Algorithms

1. Satisfaction Calculation Algorithm

- **Input:** A list of all placed buildings and their proximity to each other.
- **Process:**
 1. For each building, calculate its impact on satisfaction based on its type and location.
 2. Sum the impact of all buildings.
 3. Adjust the satisfaction based on the outcome of any active events.
- **Output:** Updated satisfaction score.

2. Event Trigger Algorithm

- **Input:** Current game time and random event timer.
- **Process:**
 1. If the timer reaches zero, trigger a random event from the `events` list.
 2. If it's time for a fixed event, trigger the specific event based on the game time.
 3. Resolve the event's impact on satisfaction and display the event to the player.
- **Output:** Updated game state and event log.

6.2 Diagrams

Here are some of the diagrams that we came up with using a Python module named Graphviz based on the algorithms:

The first diagram, Figure 1, is a class diagram showing the relationship between the grid system which represents the map where we can place and move buildings, Buildings which have a type and a position, Events which have a type and duration and can affect satisfaction and finally the student satisfaction which has one method being calculating satisfaction. The chequered arrow represents the child class of the building class whereas the second diagram, Figure 2,

Figure 1

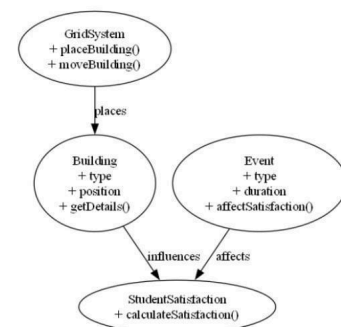
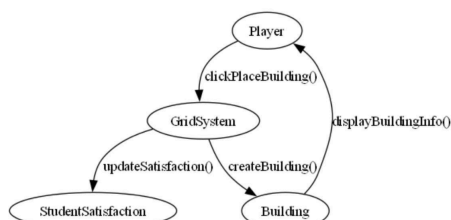


Figure 2



shows a sequence of events of how the game should function. It starts with the player who plays the game, which then places a building onto the grid system, updates the student satisfaction and then the details of the building are returned to the player.

We then updated Figure 1 using Lucid shown earlier and made it into a proper OOP class diagram, **conforming to the standards outlined by plantUML [1], which is the software used to model the class diagram.**

This diagram illustrated in Figure 3 fulfils the requirements of
 UR_INFLUENCE_SATISFACTION, FR_PLACE_BUILDING, FR_MAP,
 NFR_FUNCTIONAL.

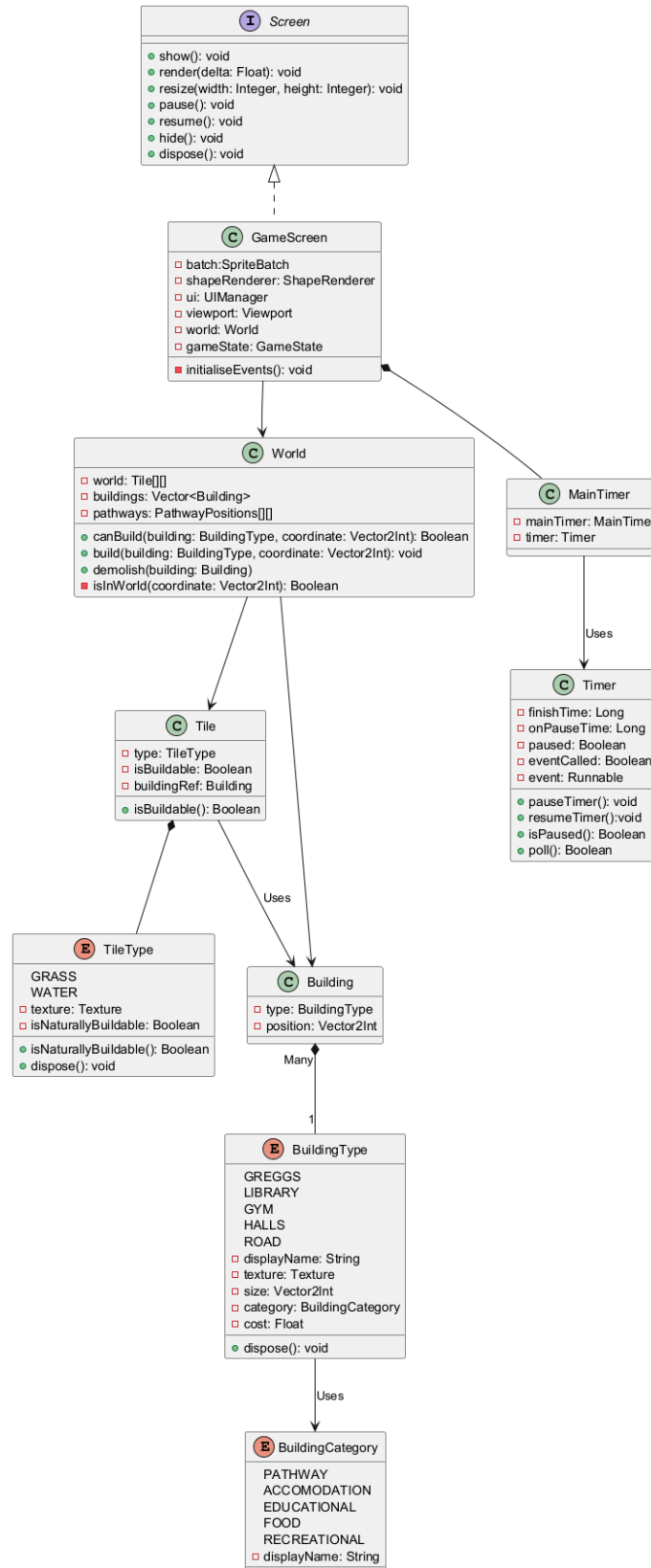


Figure 3: A class diagram representing the key functional classes in the game.

PlantUML was chosen as our UML development tool due to its simplicity and extensive documentation, facilitating the creation of extensive and accurate class diagrams. Moreover, it provides resources for alternative graph creation, such as sequence and state diagrams, which provides visual cohesion throughout our models. Other options were considered, such as LucidChart, however diagrams would have to be manually formatted, subsequently increasing both the difficulty and length of the modelling process.

PlantUML automatically formats text documents (providing they comply with the syntax expected) using a Java application which can be downloaded from the website. In addition to this, the application allows the modelling of different class types found in the Java programming language, such as Enums, Abstract and Interfaces. With these factors in mind, we felt the utility provided by plantUML made it the obvious choice for our architectural designs.

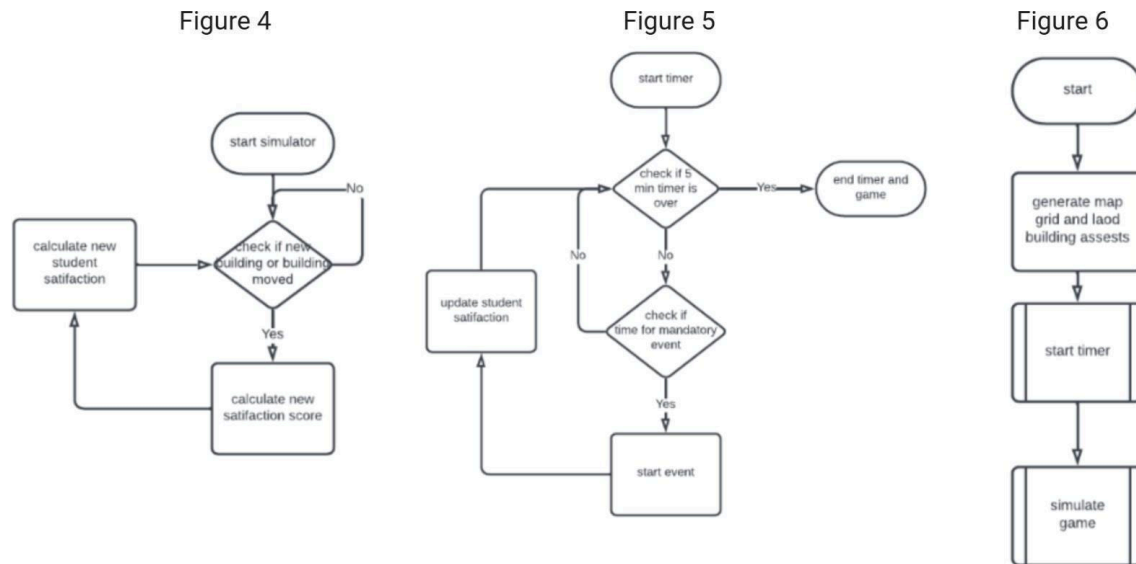
As seen from the full class diagram, the current implementation makes heavy use of the Java Enum class type. This design choice provides a multitude of benefits to the game, which are discussed below:

- **Readability:** by using Enum classes to represent structures like buildings and tiles, each constant can be given a meaningful and descriptive name. In addition, the size of the codebase is significantly reduced by utilising this class type in comparison to an Abstract class or Interface. Were these class types to be used to represent buildings and tiles, the requirement for inheritance would increase the number of classes in the game and potentially increase memory utilisation depending on the attributes and methods assigned to each instance. Whilst not a crucial implication in the current state of the game, it is nevertheless important to consider architectural choices and their effects on future development.
- **Expandability:** it is essential that our game can be maintained and expanded with ease in future iterations of development. Java Enum allows new assets to be implemented within the game with minimal effort or difficulty, as they can be listed as new constants in the class in a single line of code.
- **Consistency and Security:** Java Enum classes ensure compile-time type safety. This means variables can only take values assigned in the Enum class, mitigating any invalid type exceptions that could be thrown by using alternative class constructs and reducing the likelihood of runtime errors.

A complete class diagram for all of the game's entities can be found on our website [here](#) (figure 1).

The complete diagram illustrates the use of packages in the codebase, enhancing the readability and cohesion of the codebase. By organising similar classes in this way, the overall structure and relations between classes are easier to comprehend and model. Furthermore, using packages encourages the proper upkeep and maintenance and debugging is simplified through the grouping of classes with similar functionality.

Figures 4, 5 and 6 are flow charts that represent sequences of the game. Figure 4 accounts for the satisfaction and how it is calculated based on the building proximity; Figure 5 determines how the timer affects the events during the game and satisfaction; Figure 6 walks through the main simulation itself.



Following the creation of the full class diagram, we felt it important to provide a state diagram showcasing an abstracted view of the interactions within the system that altered the game's states. This can be seen below.

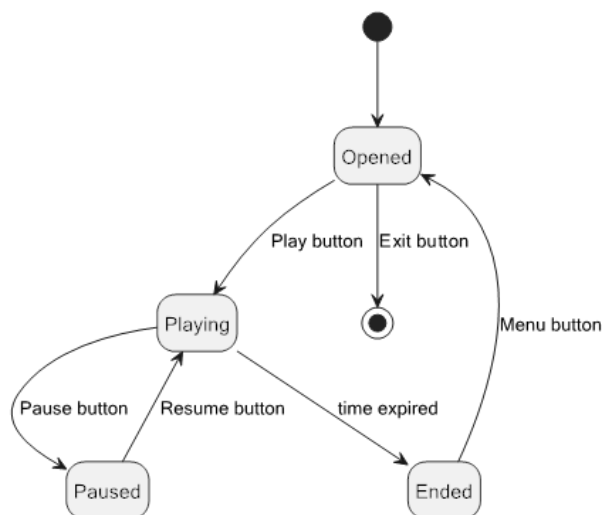


Figure 7: A plantUML state diagram depicting the various state relations in the game.

In addition to this state diagram, a sequence diagram is also provided that depicts the user's interaction with the building system implemented in the game. The building system was chosen as it reflects the core functionality within the game and therefore depicts an interaction that every user is likely to undertake and experience. This has been provided below.

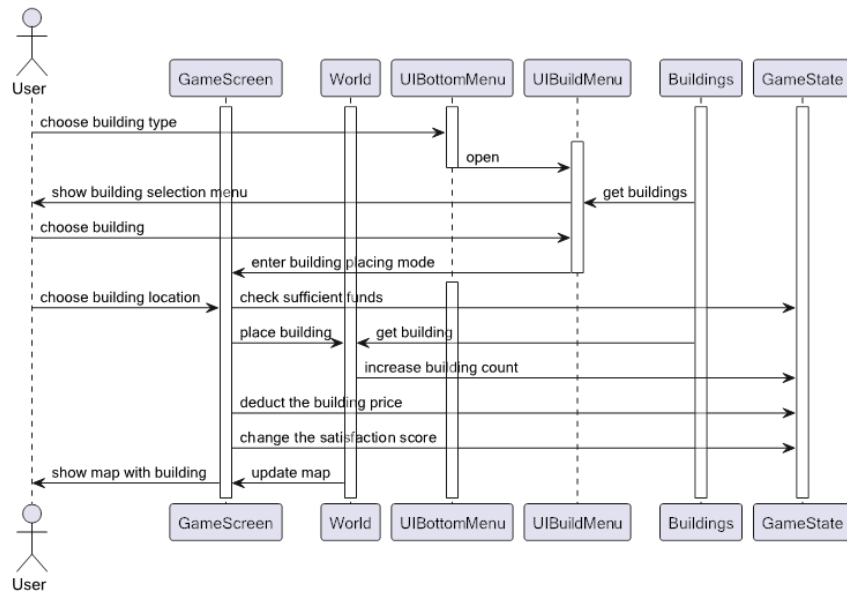


Figure 8: a plantUML sequence diagram depicting user interaction with the building system.

Subsequently, our focus turned to the implementation of the achievement and leaderboard functionalities in the design requirements. This functionality fulfils the FR_ACHIEVEMENTS and FR_LEADERBOARD requirements elicited in the [updated user requirements document](#). Upon these changes, an [updated class diagram](#) (figure 4) was created to reflect the new relations within the system. A few design considerations are discussed below:

- **Use of UIElement abstract class:** All UI elements from the first stage of design inherit and implement the methods of a single abstract class. This design choice was continued when creating UI elements for both achievements and leaderboard components to maintain consistency throughout the codebase as it is advantageous to reuse components rather than define multiple new ones.
- **Creation of IAchievement interface:** By providing a contract that all achievements must adhere to through this interface, we are able to ensure they are similar in behaviour and criteria. Moreover, a potentially vast amount of achievements and achievement types can be implemented through extension of this interface.
- **Creation of achievement types:** Our game features two achievement types, sustained and milestone achievements. These two classes implement the IAchievement interface, with the former denoting achievements the user unlocks over a certain period of time and the latter denoting achievements the user unlocks by fulfilling certain criteria. By giving users the ability to unlock achievements through different conditions, we hope to fulfil the NFR_ENJOYABLE user requirement, as these achievement types incentivise the user to challenge themselves throughout the duration of the game.
- **Creation of AchievementsManager:** All of our achievements are initialised through this class, allowing easy extension as the game progresses and grows in its development. The class is an aggregation of achievement types each with independent

logical conditions and descriptions defined separately from one another, providing the achievement system with the modularity expected in section 3 of this document.

The final required component of our system was that of event generation and randomisation, fulfilling the FR_EVENTS elicited in our user requirements document. The final class diagram comprising all functionality within the game can be found [here](#) (**figure 5**). The components of this functionality are discussed below:

- **Bias:** The concept of bias was introduced to provide a dynamic adjustment to the impact of events within the game, depending on the state of variables such as money and building count when the event is randomly triggered. We felt this would further strengthen and elevate the user experience, as the game reacts to how users choose to play the game through this bias.
- **Creation of Enum Classes:** Both RandomEvents and BiasType are Java Enum classes. As discussed previously, the use of these classes provides easy extension in later stages of development and would allow further events to be added as required. In the game's current state, five events are present. Given the ease of extension through these classes, we believed it would be beneficial to meet the requirements for the system through implementing only five events, as including any more would fall outside the scope of the project.

With this feature implemented, we are content that the architecture discussed meets the requirements for the system. Furthermore, we are confident that the structures chosen within our design will benefit subsequent stages of development.

7. Data Structures

4.1. Grid Representation

- The **Grid System** is represented as a 2D array (`Cell[][] grid`) of `Cell` objects. Each cell can hold either a building, road or be empty.
- **JSON Integration:** The grid and its contents such as the buildings and roads can be serialised into JSON format for data persistence.

4.2. Event Management

- Events are stored in a `List`, each event having attributes like `ev`.
- **JSON Integration:** Events are saved as JSON objects, allowing their state to be persisted and loaded during gameplay.

References

[1] '*Class Diagram*', plantUML.com,
<https://plantuml.com/class-diagram#:~:text=QA%2D16258%5D-,Relations%20between%20classes,-Relations%20between%20 classes> (last accessed 25th November 2024)